

LN17. 决策树和回归树

何 琨

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心

邮箱: brooklet60@hust.edu.cn

2026 年 4 月



- ① 决策树的动机
 - KNN 的启发
 - 动机与基本思想
 - 构造一颗树
- ② Impurity 函数
 - Gini Impurity (Gini 不纯度)
 - 熵 (Entropy)
- ③ ID3 算法
 - 基本思想
 - ID3 算法
- ④ CART
 - CART: 分类和回归树
- ⑤ 参数和非参数算法
 - 参数和非参数算法
 - 特殊情况讨论

- 1 决策树的动机
 - KNN 的启发
 - 动机与基本思想
 - 构造一颗树
- 2 Impurity 函数
 - Gini Impurity (Gini 不纯度)
 - 熵 (Entropy)
- 3 ID3 算法
 - 基本思想
 - ID3 算法
- 4 CART
 - CART: 分类和回归树
- 5 参数和非参数算法
 - 参数和非参数算法
 - 特殊情况讨论

在低维空间中， k 近邻分类器 (KNN) 实际上相当强大：

- 可以学习非线性决策边界
- 可以处理多分类问题

但也存在一些不足：

- KNN 使用了大量的存储空间 (因为需要存储整个训练数据)
- 拥有的训练数据越多，测试过程中 k NN 就越慢 (因为需要计算到所有训练输入点的距离)
- 需要一个好的距离度量

受 KNN 的启发

大多数数据都有一些固有的结构。

在 kNN 情况下，假设相似的输入有相似的邻居。

这意味着不同类别的数据点不是随机分布在空间中，而是出现在或多或少相同的类别分配的集群中。

尽管有高效的数据结构可以实现更快的最近邻居搜索，但分类器的最终目标只是给出准确的预测。

动机

- 假设有一个带有正类和负类标签的二元分类问题。
- 如果知道一个测试点落在一个包含 100 万个标记为正的点的簇 (cluster) 中, 那么甚至在计算到这 100 万个距离中的每个点的距离之前, 就能知道它的邻居将是正类。
- 因此, 只要知道测试点是在一个所有邻居都为正的区域中就足够了, 它的确切身份无关紧要。
- 决策树正是利用了这一点。不存储训练数据, 而是使用训练数据构建一个树结构, 递归地将空间划分为具有相似标签的区域。
- 决策树是一个非常常见并且优秀的机器学习算法, 它易于理解、可解释性强。它可作为分类算法, 也可用于回归模型。

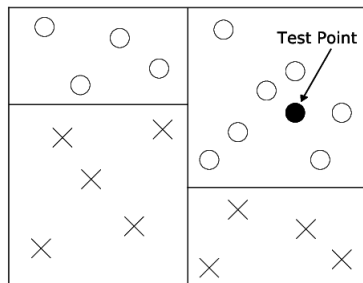
基本思想

- 树的根节点表示整个数据集。
 - 然后，该集合被一个简单的阈值 t 沿某个维度粗略地分成两半。
 - 所有具有特征值 $\geq t$ 的点都落在右子节点上，其余的点都落在左子节点上。
 - 阈值 t 和维度的选择使得生成的子节点在类成员关系方面更加纯粹。
 - 理想情况下，所有正点都落在一个子节点上，所有负点都落在另一个子节点上。
 - 如果是这样，树就完成了。如果不是，则再次拆分叶节点，直到最终所有的叶节点都是纯的（即其所有数据点包含相同的标签）或不能再拆分（在极少数情况下，两个相同的点具有不同的标签）。
-
- 基本的决策树：ID3、C4.5、CART
 - 进阶的决策树：Random Forest、Adaboost、GBDT、Xgboost 和 LightGBM。

相对于 KNN 的优势

与最近邻算法 KNN 相比，决策树有以下几个优势：

- ① 一旦构建了树，就不需要存储训练数据。相反，可以简单地存储每个标签在每个叶中有多少个点——通常这些都是纯的，所以只需要存储所有点的标签；
- ② 决策树在测试期间非常快，因为测试输入只需要沿着树向下遍历到叶子 – 预测是叶子的多数标签；
- ③ 决策树不需要度量，因为分割是基于特征阈值而不是距离。



图：二叉决策树，只存储标签

构造一颗树

目标

构造一颗树:

- ① 尽可能地紧凑
- ② 只有纯叶子节点

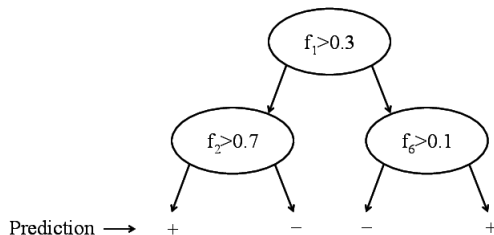


图: 二叉决策树, 只基于标签进行决策

测试

总有可能找到这样“纯”的树吗？

测试

总有可能找到这样“纯”的树吗？

A: 是的，当且仅当不存在两个输入向量具有相同特征但不同标签的情况

坏消息：

找到最小尺寸的树是 NP 难的。

好消息：

可以用贪心策略有效地近似它。通过一直分割数据来最小化 *impurity* 函数，该函数用来衡量子节点间的纯度。

构建决策树的步骤

- ① 选择根节点：计算所有特征的信息增益，选择最大的作为根节点。
- ② 递归分裂：对每个子节点重复上述过程，直到：
 - 1) 节点中样本全属于同一类别（纯度 100%）
 - 2) 所有特征已用完。
 - 3) 达到预定义的停止条件（如最大深度）。
- ③ 处理过拟合：
 - 1) 预剪枝：提前停止树的生长（设置最大深度、最小样本分裂数等）。
 - 2) 后剪枝：先让树完全生长，再自底向上合并节点。

- 1 决策树的动机
 - KNN 的启发
 - 动机与基本思想
 - 构造一颗树
- 2 Impurity 函数
 - Gini Impurity (Gini 不纯度)
 - 熵 (Entropy)
- 3 ID3 算法
 - 基本思想
 - ID3 算法
- 4 CART
 - CART: 分类和回归树
- 5 参数和非参数算法
 - 参数和非参数算法
 - 特殊情况讨论

设置

- 数据: $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, $y_i \in \{1, \dots, c\}$, 其中 c 是类的数目
- 设 $S_k \subseteq S$ 为 $S_k = \{(\mathbf{x}, y) \in S : y = k\}$ (所有输入标签为 k)。 $S = S_1 \cup \dots \cup S_c$
- 定义: $p_k = \frac{|S_k|}{|S|} \leftarrow$ 输入 S 中标签为 k 的比例

Gini Impurity (Gini 不纯度)

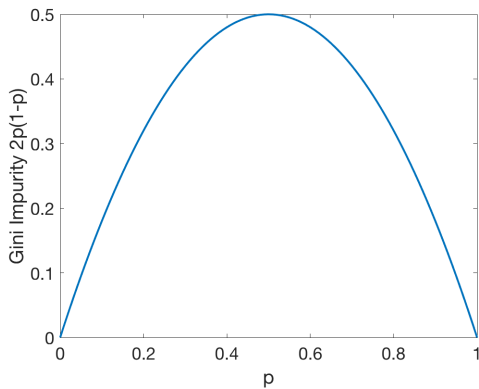
- 将 CART(分类和回归树) 算法用于分类树。
- Gini impurity(以意大利数学家 Corrado Gini 命名) 是一种度量。若根据标签在子集中的分布随机标记, 则从集合中随机选择的元素将被错误标记的频率。
- 基尼系数杂质的计算方法是将标签为 i 的数据点被选中的概率 p_i 乘以错误分类该数据点的概率 $\sum_{k \neq i} p_k = 1 - p_i$ 相乘。当节点中的所有情况都属于单个目标类别时, 该指标将达到最小值 (零)。
- 注意: 不要与 Gini Coefficient (Gini 系数) 混淆

Gini Impurity (Gini 不纯度)

决策树的一个叶子节点的 Gini Impurity 定义为:

$$G(S) = \sum_{k=1}^c p_k(1 - p_k), \quad p_k = \frac{|S_k|}{|S|}$$

Gini Impurity 的图示



二元分类时的 Gini Impurity 图示，该值在 $p = 0.5$ 时达到最大

一棵决策树的 Gini Impurity (Gini 不纯度):

$$G^T(S) = \frac{|S_L|}{|S|} G^T(S_L) + \frac{|S_R|}{|S|} G^T(S_R)$$

其中:

- $S = S_L \cup S_R$
- $S_L \cap S_R = \emptyset$
- $\frac{|S_L|}{|S|} \leftarrow$ 左子树的输入数据占比
- $\frac{|S_R|}{|S|} \leftarrow$ 右子树的输入数据占比

熵 (Entropy)

- $p_1, \dots, p_k$ 的定义同上, 即每一类的输入数据占比。
- 注意, 最差情况下为均匀分布: $p_1 = p_2 = \dots = p_c = \frac{1}{c}$
- 因为此时, 每个叶子都是等可能的。
- 预测则是随机猜测。
- 把 impurity (不纯度) 定义为与均匀的差别。=> 使用 KL 散度 (KL-Divergence) 来计算两者的“接近程度”。
- 注意: KL 散度是非对称的, 即 $KL(p||q) \neq KL(q||p)$ 。

熵 (Entropy)

令 $q_1, \dots, q_c$ 为均匀标签/分布, 即: $q_k = \frac{1}{c}, \forall k$

$$\begin{aligned} KL(p||q) &= \sum_{k=1}^c p_k \log \frac{p_k}{q_k} \geq 0 \leftarrow KL\text{-散度} \\ &= \sum_k (p_k \log(p_k) - p_k \log(q_k)), \text{ 其中 } q_k = \frac{1}{c} \\ &= \sum_k (p_k \log(p_k) + p_k \log(c)) \\ &= \sum_k p_k \log(p_k) + \log(c) \sum_k p_k, \text{ 其中 } \log(c) \leftarrow \text{常数}, \sum_k p_k = 1 \end{aligned}$$

熵 (Entropy)

由上页得: $KL(p||q) = \sum_k p_k \log(p_k) + \log(c)$

最大化 KL 散度:

$$\begin{aligned}\max_p KL(p||q) &= \max_p \sum_k p_k \log(p_k) \\ &= \min_p - \sum_k p_k \log(p_k) \\ &= \min_p H(S) \leftarrow \text{最小化熵}\end{aligned}$$

树的熵 (Entropy over Tree) :

$$H(S) = p^L H(S^L) + p^R H(S^R)$$

其中:

$$p^L = \frac{|S^L|}{|S|}, p^R = \frac{|S^R|}{|S|}$$

- 1 决策树的动机
 - KNN 的启发
 - 动机与基本思想
 - 构造一颗树
- 2 Impurity 函数
 - Gini Impurity (Gini 不纯度)
 - 熵 (Entropy)
- 3 ID3 算法
 - 基本思想
 - ID3 算法
- 4 CART
 - CART: 分类和回归树
- 5 参数和非参数算法
 - 参数和非参数算法
 - 特殊情况讨论

基本思想

- ID3 算法是建立在奥卡姆剃刀（用较少的东西，同样可以做好事情）的基础上：越是小型的决策树越优于大的决策树。
- 信息论：信息熵越大，样本纯度越低。
- ID3 算法的核心思想就是以信息增益来度量特征选择，选择信息增益最大的特征进行分裂。
- 算法采用自顶向下的贪婪搜索遍历可能的决策树空间。

ID3 基础算法

ID3(S):

$\begin{cases} \text{若 } \exists \bar{y} \text{ s.t. } \forall (x, y) \in S, y = \bar{y} \Rightarrow \text{return 有标签 } \bar{y} \text{ 的叶子} \\ \text{若 } \exists \bar{x} \text{ s.t. } \forall (x, y) \in S, x = \bar{x} \Rightarrow \text{return 具有 mode}(y : (x, y) \in S) \text{ 或 均值 (回归) 的叶子} \end{cases}$

由上式可知，ID3 算法在以下两种情况下停机：

- ① 当前子集中的所有数据点均有相同的标签：则停止拆分子集，并新增一个标签为 y 的叶子。
- ② 没有更多的属性可以用来划分当前子集：则新增一个叶子，并用 S 中出现频率最高的 y 来标记它。

- 尝试所有的特征和所有可能的划分。
- 选择使 impurity(不纯度) 最小化的划分 (如: $x_f > t$), 其中: $f \leftarrow$ 特征和 $t \leftarrow$ 阈值。
- 递归地定义:

$$\begin{cases} S^L = \{(x, y) \in S : x_f \leq t\} \\ S^R = \{(x, y) \in S : x_f > t\} \end{cases}$$

- ① 初始化特征集合和数据集合；
- ② 计算数据集合信息熵和所有特征的条件熵，选择信息增益最大的特征作为当前决策节点；
(信息增益 = 信息熵 - 条件熵)
- ③ 更新数据集合和特征集合（删除上一步使用的特征，并按照特征值来划分不同分支的数据集合）；
- ④ 重复 2, 3 两步，若子集值包含单一特征，则为分支叶子节点。

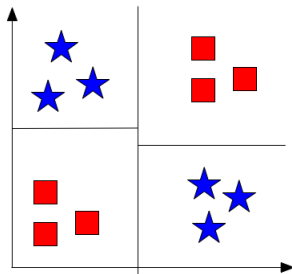
提问

若当前没有任何划分可以改进 impurity 时，为什么不停机呢？

测试

若当前没有任何划分可以改进 impurity 时，为什么不停机呢？

举例回答: XOR

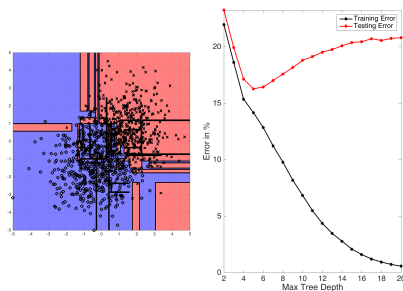


- 第一次划分不会改进 impurity，但继续下去，第二次划分可以。
- 决策树是短视的（即贪心策略）。

ID3 的缺点：没有剪枝策略，容易过拟合；

- 没有剪枝策略，容易过拟合；
- 信息增益准则对可取值数目较多的特征有所偏好，类似“编号”的特征其信息增益接近于 1；
- 只能用于处理离散分布的特征；
- 没有考虑缺失值。

ID3 的过拟合问题



随着树深度的增加，ID3 树容易出现过拟合问题。

- 左图展示了从两个高斯分布绘制的二分类数据集上学习到的决策边界。
- 右图展示了随着树深度增加，其测试和训练误差的变化。

- ① 决策树的动机
 - KNN 的启发
 - 动机与基本思想
 - 构造一颗树
- ② Impurity 函数
 - Gini Impurity (Gini 不纯度)
 - 熵 (Entropy)
- ③ ID3 算法
 - 基本思想
 - ID3 算法
- ④ CART
 - CART: 分类和回归树
- ⑤ 参数和非参数算法
 - 参数和非参数算法
 - 特殊情况讨论

CART(Classification and Regression Trees): 分类和回归树, 其输入和预测特征既可以是连续型的也可以是离散型的

- 设标签是连续的: $y_i \in \mathbb{R}$
- 使用 Gini 系数作为变量的不纯度量
- 均方损失

$$L(S) = \frac{1}{|S|} \sum_{(x,y) \in S} (y - \bar{y}_S)^2 \leftarrow \text{与平均标签的均方误差}$$

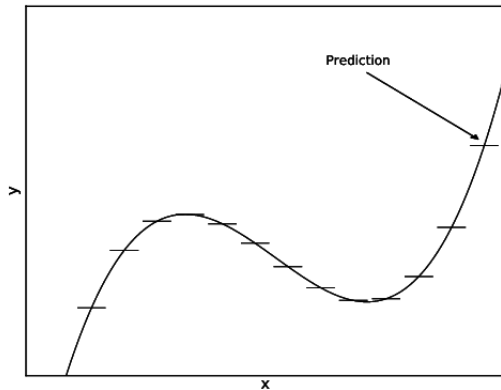
其中: $\bar{y}_S = \frac{1}{|S|} \sum_{(x,y) \in S} y \leftarrow \text{平均标签}$

- 在叶子节点, 预测 $\bar{y}_S$, 找到最佳划分只需要 $O(n \log n)$ 的时间成本。

CART 的基本过程有分裂、剪枝和树选择:

- **分裂**: 分裂过程是一个二叉递归划分过程, CART 没有停止准则, 会一直生长下去;
- **剪枝**: 采用代价复杂度剪枝, 从最大树开始, 每次选择训练数据熵对整体性能贡献最小的那个分裂节点作为下一个剪枝对象, 直到只剩下根节点。
CART 会产生一系列嵌套的剪枝树, 需要从中选出一颗最优的决策树;
- **树选择**: 用单独的测试集评估每棵剪枝树的预测性能 (也可以用交叉验证)。

CART 的图示



CAR 方法 T 总结:

- CART 是轻量级的分类器
- 测试时, 非常快
- 通常在准确性上没有竞争力, 但可以通过 bagging(随机森林) 和 boosting(梯度增强树) 变得非常强大

- ① 决策树的动机
 - KNN 的启发
 - 动机与基本思想
 - 构造一颗树
- ② Impurity 函数
 - Gini Impurity (Gini 不纯度)
 - 熵 (Entropy)
- ③ ID3 算法
 - 基本思想
 - ID3 算法
- ④ CART
 - CART: 分类和回归树
- ⑤ 参数和非参数算法
 - 参数和非参数算法
 - 特殊情况讨论

至此，我们已经学习了很多种算法。

可以将它们分为不同的类别，例如：

生成性 vs. 判别式或概率 vs. 非概率。

接下来，我们将详细讨论参数 vs. 非参数算法。

- 参数算法是一种具有常量参数集的算法，该参数集的大小与训练样本的数量无关。
- 可以把它理解成：需要存储经过训练的分类器所需的空间大小。
- 参数算法的例子：感知器算法，逻辑回归。它们的参数包括 w, b ，它定义了分离的超平面。 w 的维数取决于训练数据的维数，但与训练样本的数量无关。

- 相比之下，非参数算法的参数量是训练样本的函数。
- 非参数算法的一个例子是 k -NN 分类器。在训练期间，需要存储整个训练数据——因此我们学习的参数与训练集相同，并且参数的数量 (/所需要的存储) 随着训练集的大小线性增长。

一个有意思的边界情况是核支持向量机。它很大程度上取决于我们使用的是哪个核。

- 线性支持向量机是参数算法 (原因与感知机或逻辑回归相同)。如果核是线性, 就是参数算法。
- 然而, 如果使用 RBF 核, 那么就不能表示有限维超平面的分类器。相反, 必须存储支持向量及其对应的对偶变量 α_i ——其数量是数据集大小 (和复杂性) 的函数。因此, 具有 RBF 核的支持向量机是非参数的。
- 一个特别的介于两者之间的情况是多项式核。

它代表了一个极高但仍然有限维空间中的超平面。

从实现上讲, 可以将具有多项式核的支持向量机的任何解表示为具有固定数量参数的极高维空间中的超平面, 因此, 该算法 (在实现上) 是参数的。

然而, 在实践中, 这是不现实的。相反, 存储支持向量和它们对应的对偶变量几乎总是更经济的 (就像 RBF 核一样)。

因此, 它在实现上是参数的, 但表现得像一个非参数算法。

决策树也是一个有意思的例子。

- 如果它们被训练到全深度，则是非参数的，因为决策树的深度是训练数据的函数（实际上是 $O(\log_2(n))$ ）。
- 如果将树深度限制到某个阈值，它们就成为参数算法（因为在观察训练数据之前，模型大小的上限就已经知道了）。

The End